

FSA v1.5 — purely functional Julia bridge:  
open-loop v1 dynamics + closed-loop SMC<sup>2</sup>-MPC,  
written for human and agent comprehension and Lean4  
portability

Coding session writeup

2026-05-08

**Abstract**

This document is the design specification for FSA v1.5, a third Julia codebase sitting between v1 (open-loop only, imperative) and v2 (closed-loop with multi-channel observations, imperative). v1.5 re-implements the closed-loop SMC<sup>2</sup>-MPC pipeline in a purely functional, immutable-data style on top of v1’s dynamics and GPU controller cost kernel. The goals are: (i) clarity — humans and agents should be able to read each public function and understand its semantics in isolation; (ii) Lean4 portability — pattern-matching sites are written in a style that maps near-mechanically to Lean4’s `match ... with` for future formal verification; (iii) end-to-end validation — v1.5 reproduces v1’s reference 28-day qualitative behaviour with a small, transparent pipeline.

## Contents

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                       | <b>2</b>  |
| <b>2</b> | <b>Repository layout</b>                                                  | <b>2</b>  |
| <b>3</b> | <b>Functional design principles</b>                                       | <b>3</b>  |
| <b>4</b> | <b>Per-file semantic specification</b>                                    | <b>4</b>  |
| 4.1      | <code>_dynamics.jl</code> (v1 verbatim)                                   | 4         |
| 4.2      | <code>simulation.jl</code> (NEW)                                          | 4         |
| 4.3      | <code>_plant.jl</code> (NEW, written from first principles)               | 5         |
| 4.4      | <code>gpu_control.jl</code> (v1 verbatim)                                 | 6         |
| 4.5      | <code>estimation.jl</code> (NEW)                                          | 6         |
| 4.6      | <code>gpu_pf.jl</code> (NEW)                                              | 7         |
| 4.7      | <code>FSAHighRes.jl</code> (NEW)                                          | 8         |
| <b>5</b> | <b>Data flow and computational graph</b>                                  | <b>8</b>  |
| 5.1      | One closed-loop stride iteration                                          | 8         |
| 5.2      | Filter dataflow (single <code>gpu_log_density</code> call)                | 9         |
| 5.3      | Controller dataflow (one <code>controller_plan</code> call)               | 10        |
| <b>6</b> | <b>CLI flags reference for <code>bench_smc_full_mpc_fsa_gpu.jl</code></b> | <b>11</b> |
| 6.1      | Bench / filter flags                                                      | 11        |
| 6.2      | Controller flags (CLI-exposed, formerly hardcoded)                        | 11        |
| 6.3      | Example invocations                                                       | 12        |

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>7 Lean4-port mapping appendix</b>              | <b>12</b> |
| 7.1 Side-by-side mappings                         | 12        |
| 7.2 Type-system gotchas for the future Lean4 port | 14        |
| 7.3 Verification of the port (when it happens)    | 14        |

## 1 Introduction

Three FSA Julia codebases now coexist:

- **v1** (`version_1_Julia/models/fsa_high_res/`) — G0 dynamics (no reparametrisation), GPU cost kernel, open-loop SMC<sup>2</sup> control. No plant. No filter. No closed loop.
- **v2** (`version_2_Julia/models/fsa_high_res/`) — G1 reparametrised dynamics, four-channel observation model (HR/sleep/stress/steps), `StepwisePlant`, GPU particle filter, full closed-loop SMC<sup>2</sup>-MPC bench. Imperative, with `mutable struct` and `!`-suffix mutators throughout. Documented in `version_2_Julia/docs/julia_fsa_writeup.pdf`.
- **v1.5** (`version_1_5_Julia/`, *this writeup*) — a clean, purely functional bridge. Reuses v1’s `_dynamics.jl` and `gpu_control.jl` verbatim. Adds the simplest possible plant, filter, and closed-loop bench. Written in Julia idioms with `Match.jl` pattern matching at every site that will need pattern matching in a future Lean4 port.

**Status (validated).** The v1.5 build cleared the FIM gate at rank 7/7,  $\kappa = 2.95 \times 10^7$  on the realistic 14-day window after pinning the four weakly-identified parameters  $\{\tau_B, \eta, \varepsilon_A, \mu_{FF}\}$  at truth. The 28-day open-loop bench reproduces v1’s reference qualitative behaviour: *A* dips from 0.10 to 0.06 around day 7–10 and climbs to 0.16 by day 28. The 28-day closed-loop bench under a strengthened controller (`ctrl-n-smc=256`, `ctrl-n-inner=64`, `ctrl-num-mcmc=8`, `ctrl-hmc-leap=16`) produces the same A-dip-then-climb pattern with *A* finishing at 0.154.

**Why purely functional.** v1 and v2 are imperative Python ports: `mutable struct StepwisePlant`, `!`-suffix in-place mutators, history dicts `push!`-ed onto, RNG state threaded as a stateful `MersenneTwister`. These are tangled and hard for new readers (human or agent) to follow. v1.5 is a clean redesign in functional, stateless Julia. Performance is explicitly not a goal — clarity and composability are. Where the GPU kernel itself must mutate (as `KernelAbstractions` kernels do), v1.5 wraps the mutation behind a pure facade so callers never see it.

**Why Match.jl.** The criterion for using `@match` in v1.5 is *not* “does this improve readability today?”. It is “will the Lean4 version of this site use `match ... with?`”. Lean4’s native pattern matching is structurally near-identical to `Match.jl`’s, so writing the Julia in `@match` idioms now makes the future Julia → Lean4 port nearly mechanical.

## 2 Repository layout

```
version_1_5_Julia/
  Project.toml           # +Match.jl
  docs/
    julia_fsa_v15_writeup.tex    # this document
    julia_fsa_v15_writeup.pdf
  models/fsa_high_res/
    _dynamics.jl           # v1 verbatim --- G0 SDE
    simulation.jl          # NEW --- pure, owns BINS_PER_DAY,
```

```

                                #          params, obs sampler, adapters
_plant.jl                      # NEW --- pure PlantState + plant_step
                                #          + plant_rollout (no '!')
gpu_control.jl                 # v1 verbatim --- GPU cost kernel
estimation.jl                  # NEW --- pure propagate + obs_log_weight
gpu_pf.jl                     # NEW --- pure facade over the
                                #          segmented-PF kernel
FSAHighRes.jl                  # NEW --- module aggregator
tools/
  fim_check.jl                 # FIM gate (HARD GATE per plan)
  psim_sanity.jl               # plant rollout sanity test
  test_gpu_pf.jl               # 6-test smoke suite for gpu_pf.jl
  bench_smc_full_mpc_fsa_gpu.jl # closed-loop bench (foldl-based)
outputs/fsa_high_res/          # auto-created at run time

```

### 3 Functional design principles

v1.5 commits to five rules. Each rule is enforced by audit; no public API in v1.5 violates them.

1. **No mutable structs in the public API.** v1.5 has exactly one struct, `PlantState`, with two fields: an `SVector{3, Float64}` (immutable, stack-allocated) and an `Int`. State updates produce a NEW `PlantState` value, not a mutation of the old one. Compare to v2's mutable struct `StepwisePlant` with mutable `state::Vector{Float64}`, `t_bin::Int`, and `history::Dict{Symbol, Vector{Any}}`.
2. **No !-suffix mutating methods.** Every public function returns new values. `plant_step(state,  $\Phi_t$ , params, dt, key)` returns `(state=new_state, obs=...)`; the caller can build up history by collecting returned values, or by `accumulate/foldl` over time bins. Compare to v2's `advance_subdaily!(plant,  $\Phi$ )` which pushes onto `plant.history[:trajectory]`.
3. **Explicit RNG keys, JAX-style.** Pass an explicit `key::UInt64` to every randomised function; the function takes the key and returns a deterministic result. Internal split-keys are derived deterministically from the input key via `hash((key, :sub_purpose))`. Two callers with the same `(state, key)` get the same answer, period. Compare to v2's threaded `rng::MersenneTwister` which mutates as `randn` is called.
4. **Pure facade over the inherently imperative GPU kernel.** `KernelAbstractions` kernels write into preallocated `CuArrays`. v1.5 wraps every kernel-touching public function in a pure facade: `gpu_log_density(target, U, grid_obs, key) → Vector{Float64}` allocates a fresh output, dispatches the kernel, returns. Mutation is fully encapsulated; callers never see the `CuArray`.
5. **I/O isolated to the bench's top-level main().** JLD2 and PNG writes happen exactly once at the end of the bench. Logging happens at the bench level, not inside `plant_step` / `propagate` / `obs_log_weight`.

|                                     | v1 / v2 (imperative)                                                     | v1.5 (functional)                                                                                               |
|-------------------------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Imperative → functional name table. | <code>mutable struct StepwisePlant</code>                                | <code>struct PlantState (im</code>                                                                              |
|                                     | <code>advance_subdaily!(plant, Φ)</code>                                 | <code>plant_rollout(state,</code><br><code>turning (final_state,</code><br><code>traj returned, accumula</code> |
|                                     | <code>plant.history[:traj]  &gt; vcat</code>                             | <code>dropped — grid_obs is</code>                                                                              |
|                                     | <code>update_window_obs!(target, obs)</code>                             | <code>parallel_hmc_one_move</code>                                                                              |
|                                     | <code>parallel_hmc_one_move!(U, target,</code><br><code>...)</code>      | <code>key) → (new_U, n_acc)</code>                                                                              |
|                                     | <code>MersenneTwister(seed) ;</code>                                     | <code>key::UInt64 threaded c</code>                                                                             |
|                                     | <code>randn(rng, ...)</code>                                             |                                                                                                                 |
|                                     | <code>for s in 1:n; advance!(plant,</code><br><code>...) ; ...end</code> | <code>foldl((acc, s) → step</code><br><code>init=acc0)</code>                                                   |

## 4 Per-file semantic specification

This section is the contract for each public function in `models/fsa_high_res/`. Each entry gives: purpose, inputs, outputs, invariants, failure modes.

### 4.1 `_dynamics.jl` (v1 verbatim)

G0 SDE drift, state-dependent diffusion, and a multi-substep Euler–Maruyama helper. Verbatim copy of `version_1_Julia/models/fsa_high_res/_dynamics.jl` — the only file in v1.5 reused unchanged from v1.

#### Public exports.

- `TRUTH_PARAMS :: NamedTuple` — 14 truth dynamics parameters in *v1 form*: `tau_B`, `tau_F`, `kappa_B`, `kappa_F`, `epsilon_A`, `lambda_A`, `mu_0`, `mu_B`, `mu_F`, `mu_FF`, `eta`, `sigma_B`, `sigma_F`, `sigma_A`.
- `drift(y, params, Φt) :: Vector{Float64}` — pure. The G0 drift:

$$\begin{aligned}
\mu(B, F) &= \mu_0 + \mu_B B - \mu_F F - \mu_{FF} F^2, \\
dB/dt &= \kappa_B (1 + \varepsilon_A A) \Phi - B/\tau_B, \\
dF/dt &= \kappa_F \Phi - (1 + \lambda_A A)/\tau_F \cdot F, \\
dA/dt &= \mu \cdot A - \eta A^3.
\end{aligned}$$

No near-equal subtraction, no fp32 cancellation hot-spots.

- `diffusion_state_dep(y, params)` — pure. Diagonal state-dependent diffusion:  $\sigma_B \sqrt{B(1-B)}$ ,  $\sigma_F \sqrt{F}$ ,  $\sigma_A \sqrt{A}$ .
- `em_step_substepped(y, params, noise, Φt, dt; n_substeps=4)` — pure substepped Euler–Maruyama:  $n$  drift-only substeps then one diffusion increment at the outer-bin boundary. Boundary reflection:  $B \rightarrow -B$  if  $B < 0$ ,  $B \rightarrow 2 - B$  if  $B > 1$ ,  $F, A \rightarrow |\cdot|$ . Used by the plant, not by the inner PF (which embeds an analogous step inside the kernel).

**Invariants.** Pure. Same inputs → same output. Does not allocate persistent state. Generic on the value type so `ForwardDiff` can dual-number through.

### 4.2 `simulation.jl` (NEW)

Owns all bench-wide constants and parameter adapters. The most important file in v1.5 to understand because every other file imports its constants.

### Resolved at module load.

- `BINS_PER_DAY = (60 · 24) ÷ _STEP_MIN`, where `_STEP_MIN = parse(Int, get(ENV, "FSA_STEP_MINUTES", "60"))`. The env-var hook must be set BEFORE the module imports, or the constant defaults to 60 min ( $\Rightarrow$  24 bins/day).
- `DT_BIN_DAYS = 1.0 / BINS_PER_DAY`.

### Constants.

- `DEFAULT_PARAMS :: Dict{Symbol, Float64}` — 14 dynamics parameters in **v1.5 basis** ( $\kappa_B$  replaced by  $B_\infty = \kappa_B \cdot \tau_B$ ,  $\kappa_F$  replaced by  $F_\infty$ ), plus three pinned obs-noise parameters  $\sigma_{B,\text{obs}} = \sigma_{F,\text{obs}} = \sigma_{A,\text{obs}} = 0.005$ .
- `INIT_STATE = (B = 0.05, F = 0.30, A = 0.10)` — canonical initial state. Same as v1 and v2.
- `PINNED_PARAMS :: Dict{Symbol, Float64}` — four parameters NOT estimated by the filter, pinned at truth per the FIM-gate decision:  $\tau_B, \eta, \varepsilon_A, \mu_{FF}$ .

### Public functions.

- `sample_obs_bfa(state::SVector{3, Float64}, params::Dict{Symbol, Float64}, key::UInt64) -> NamedTuple{(:obs_B, :obs_F, :obs_A), Tuple{Float64, Float64, Float64}}`. Pure. Sample one Gaussian obs per latent. Same `(state, key)` always returns the same triple.
- `params_v15_to_v1_nt(p) -> NamedTuple` — basis rotation adapter. Takes a v1.5-form container ( $B_\infty, F_\infty + 9$  other dynamics params) and returns a v1-form `NamedTuple` (`kappa_B = B_inf / tau_B`, `kappa_F = F_inf / tau_F`, others unchanged). Generic on the value type — ForwardDiff dual-numbers through unchanged. Two methods (`Dict` and `NamedTuple`) collapsed into a single `@match` block (see §7).
- `fill_pinned_nt(estimated::NamedTuple) -> NamedTuple` — merges a 10-field estimated `NamedTuple` with the 4 pinned values from `PINNED_PARAMS`, returning a 14-field v1.5 `NamedTuple` ready to be passed to `params_v15_to_v1_nt`. Generic on element type.

**Why two adapters.** v1's `drift` expects a 14-field `NamedTuple` in v1 form (`kappa_B, kappa_F, ...`). The filter estimates 10 fields in v1.5 form (`tau_F, B_inf, F_inf, ...`) plus 3 diffusions. The pipeline at every drift call site is: `estimated NamedTuple`  $\rightarrow$  `fill_pinned_nt`  $\rightarrow$  14-field v1.5  $\rightarrow$  `params_v15_to_v1_nt`  $\rightarrow$  v1 form  $\rightarrow$  `drift`. Both adapters are pure.

### 4.3 \_plant.jl (NEW, written from first principles)

The plant is a pair of pure functions plus an immutable state record. No mutable `struct`, no history dict, no `!-`suffix.

#### Immutable state.

```
1 struct PlantState
2     bfa::SVector{3, Float64}      # (B, F, A) -- stack-allocated
3     t_bin::Int                   # global bin counter
4 end
```

## Public functions.

- `init_plant_state(; init_state = INIT_STATE)` — build a fresh `PlantState` at the canonical initial values.
- `plant_step(s,  $\Phi_t$ , params, dt, key) -> (state, obs)` — one Euler–Maruyama step under control  $\Phi_t$  from state  $s$ . The body: drift + state-dep diffusion + boundary reflection + obs sample. Pure: same inputs always return the same outputs. Internal RNG split: `StableRNG(key)` for the SDE noise, `hash((key, :obs))` sub-key for the obs sample.
- `plant_rollout(s0,  $\Phi_{\text{subdaily}}$ , params, dt, key0) -> NamedTuple` — apply `length( $\Phi_{\text{subdaily}}$ )` consecutive `plant_steps` from  $s_0$ . Returns `(final_state, trajectory::Matrix{Float64}(n, 3), obs_B, obs_F, obs_A, Phi)`. Pure. The internal loop builds two preallocated Vectors and writes into them, but the function as a whole is referentially transparent — the local mutation does not leak past the function boundary.

**Internal helper.** `_reflect_unit(x)` — pure:  $x < 0 \Rightarrow -x$ ;  $x > 1 \Rightarrow 2 - x$ ; otherwise  $x$ . Currently a ternary; the `@match` version (see §7) will make the cases explicit and maps 1:1 to Lean4’s `match ... with`.

## 4.4 `gpu_control.jl` (v1 verbatim)

GPU cost kernel for the controller’s RBF-decoded  $\Phi$  schedule. Verbatim copy of v1’s `gpu_control.jl`. Eq. 37 cost:  $J(\Phi) = - \int A dt + \lambda_F \int \max(F - F_{\max}, 0)^2 dt$ . No  $\lambda_\Phi \int \Phi^2$  penalty (that lived in v1’s CPU `control.jl`, which v1.5 deliberately does not copy).

## Public exports.

- `FSAv1ControlGPUPTarget` — mutable struct (this is one exception to v1.5’s purity convention; the struct is constructed by the bench at each replan, not mutated by callers).
- `gpu_cost_log_density_batched(target, theta_unc) -> Vector{Float64}` — per-chain  $-J$ . The bench wraps this in `make_log_density_fn(target)` for the framework’s `run_tempered_smc_gpu`.
- `make_log_density_fn(target) -> Function` — closure factory for the framework.

**Why kept verbatim.** v1’s `fsa_v1_cost_kernel!` is a fp32 hot-path `KernelAbstractions` kernel. Rewriting it in functional style would lose the per-thread register allocation and add overhead. The kernel is wrapped behind the bench’s `controller_plan` pure facade, which constructs a fresh target per replan and discards it after the call.

## 4.5 `estimation.jl` (NEW)

The filter side. Three pure pieces, two read-only constants.

## Constants.

- `PARAM_NAMES :: Vector{Symbol}` — the 10 estimated parameters in filter-vector order: `tau_F`, `B_inf`, `F_inf`, `lambda_A`, `mu_0`, `mu_B`, `mu_F`, `sigma_B`, `sigma_F`, `sigma_A`. The 4 pinned dynamics (`tau_B`, `eta`, `epsilon_A`, `mu_FF`) are NOT here. Neither are the 3 obs-noise (those are constants).
- `PARAM_PRIOR_CONFIG :: Vector{(Symbol, Symbol, Float64, Float64)}` — one tuple per estimated param: `(name, kind,  $\mu$ ,  $\sigma$ )` where `kind`  $\in \{:\text{LogNormal}, :\text{Normal}\}$  and  $(\mu, \sigma)$  are in unconstrained space. All 10 are `LogNormal` in v1.5.

## Public functions.

- `propagate(particles::Matrix{Float64},  $\Phi_t$ , params, dt, key) -> Matrix{Float64}` — pure prior-predictive. From each particle's  $(B, F, A)$ , run the v1 G0 SDE forward by one bin under control  $\Phi_t$ . Returns a NEW particles matrix; does not mutate the input. No locally-guided proposal — v1.5's obs is so informative (direct Gaussian on each latent every bin) that prior-predictive does not degenerate.
- `obs_log_weight(particles, obs::NamedTuple, params) -> Vector{Float64}` — pure. Sum of three diagonal Gaussian log-pdfs:

$$w_m = \log \mathcal{N}(y_B; B_m, \sigma_{B,\text{obs}}^2) + \log \mathcal{N}(y_F; F_m, \sigma_{F,\text{obs}}^2) + \log \mathcal{N}(y_A; A_m, \sigma_{A,\text{obs}}^2)$$

Allocates one fresh result vector per call.

## 4.6 gpu\_pf.jl (NEW)

The pure facade over the inherently imperative segmented-PF GPU kernel. The KernelAbstractions `@kernel` function `propagate_segment_kernel!` is private (mutating); the public API allocates fresh outputs every call.

### Immutable target.

```

1 struct FSAGPUTarget
2     K_per_chain::Int; M_max::Int; T_steps::Int; R::Int
3     n_params_v1::Int # = 14
4     n_states::Int    # = 3
5     dt::Float32; sqrt_dt::Float32
6     a_shrink::Float32; ot_max_weight::Float32; ...
7     inv_2sigma_B^2::Float32; ...; log_norm_obs::Float32
8     params_per_chain::CuArray{Float32, 2} # (M_max, 14)
9     Phi_seq::CuArray{Float32, 1}; ...obs buffers
10    noise_grid::CuArray{Float32, 3}      # CRN
11    bufs::GPUSegmentedBuffers            # framework
12    propagate_kernel::Any                 # KA handle
13 end

```

The CuArrays are GPU-resident buffers preallocated at construction. They are mutable from the GPU's perspective (kernels write to them) but no public API mutates the struct fields. The same `target` is returned for the same constructor inputs.

## Public functions.

- `FSAGPUTarget(; K_per_chain, M_max, T_steps, R=4, dt, ...)` — constructor. Allocates GPU buffers, computes  $1/(2\sigma_X^2)$  constants from `DEFAULT_PARAMS`, draws the CRN noise grid from `noise_seed`.
- `gpu_log_density(target, U_unc::Matrix{Float64}(M, 10), grid_obs::NamedTuple, key::UInt64) -> Vector{Float64}` — pure facade. M chains in unconstrained space; for each, the pipeline is  $u \rightarrow \text{\_to\_v15\_constrained\_nt} \rightarrow \text{\_fill\_pinned\_nt} \rightarrow \text{\_params.v15\_to\_v1\_nt} \rightarrow \text{\_pack\_v1\_row!} \rightarrow$  GPU kernel. Output is a fresh `Vector{Float64}` of length M.
- `gpu_grads(target, U_unc, grid_obs, h, key) -> (vals, grads)` — central-difference FD gradient. Builds the  $M \cdot (1+2d)$  perturbation matrix functionally, calls `gpu_log_density` once on the full block, central-diffs.
- `parallel_hmc_one_move(U, target, grid_obs,  $\varepsilon$ , L, prior_means, prior_sigmas, key) -> (U_new, n_acc)` — one parallel-chains HMC move with FD-gradient leapfrog. Returns a NEW `U_new` matrix; does not mutate `U`. Acceptance is per-chain Metropolis with key-derived uniform draws.

## Internal helpers.

- `_to_v15_constrained_nt(u)` — 10-element `u_unc`  $\rightarrow$  NamedTuple of 10 LogNormal-constrained values via `exp(clamp(u.i, -20, 20))`.
- `_pack_v1_row!(row, p_v1)` — write 14 fields of a v1-form NamedTuple into a Float32 row. The only `!`-suffix helper in v1.5; private and called only within the pure facade.
- `propagate_segment_kernel!` — the GPU kernel. Mutating by design (KernelAbstractions semantics). Inputs: `log_w_inout (M*K,)`, `particles_in (M*K, 3)`, `params_per_chain (M, 14)`, `Phi_seq (T,)`, `obs_{B,F,A}_value (T,)`, `noise_grid (K, T, 3)`. Body: for each thread (one (chain, particle)) and each of  $R$  bins in the segment: drift + diffusion + boundary reflection + 3-channel Gaussian log-weight.

## 4.7 FSAHighRes.jl (NEW)

Module aggregator. Includes the six other files in load order (`_dynamics.jl`, `simulation.jl`, `_plant.jl`, `gpu_control.jl`, `estimation.jl`, `gpu_pf.jl`) and re-exports their public names.

# 5 Data flow and computational graph

## 5.1 One closed-loop stride iteration

The bench's `stride_step` is a pure function from `(acc, stride_idx)` to a new `acc'`. The full 14-day or 28-day bench is `foldl(stride_step, 1:n.strides; init=acc0)`, with the only I/O being a JLD2 dump and PNG generation at the end.

The per-stride flow:

```
acc.plan_phi[offset+1 : offset+stride_bins]    # slice next stride's Phi
|
v
plant_rollout(acc.plant_state, phi, DEFAULT_PARAMS, dt, key)
|   PURE | returns (final_state, trajectory, obs_B, obs_F, obs_A, Phi)
v
new_obs ← (vcat acc.obs_history p.obs_*)        # immutable concat
new_traj ← (vcat acc.traj_history p.trajectory)
|
v
hist_end < window_bins ?
| YES                                     | NO
v                                     v
skip filter                             window_grid_obs(...) PURE
xhat = p.final_state                     |
                                         v
                                         run_outer_smc(target, grid_obs, n_smc, ...,
                                                         init_particles=acc.filter_post,
                                                         key)                PURE FACADE
                                         |
                                         v
                                         filter_out.U_post (M, 10)
                                         |
                                         v
                                         extract_xhat(target, n_smc)    reads bufs
                                         |
```



```

                                v
                                new_xhat ← SVector{3, Float64}
                                |
                                v
!is_open_loop && stride_idx % replan_K == 0 ?
                                | YES                                | NO
                                v                                    v
posterior_mean_v15(new_filter_post)                                keep acc.plan_phi
                                |                                    |
                                v                                    |
fill_pinned_nt(...) → params_v15_to_v1_nt(...)                    |
                                |                                    |
                                v                                    |
controller_plan(params_v1, new_xhat, H_plan_bins,                  |
                ctrl_cfg, hash((seed, :ctrl, stride_idx)))        |
                                | FRAMEWORK CALL | run_tempered_smc_gpu |
                                v                                    |
new_plan, offset=0                                                |
                                |                                    |
                                v                                    v
return new acc' = (plant_state=p.final_state, plan_phi=...,
                  filter_post=new_filter_post,
                  traj_history=new_traj, obs_history=new_obs,
                  per_stride_log=...)

```

## 5.2 Filter dataflow (single gpu\_log\_density call)

```

U_unc :: Matrix{Float64}(M, 10)      # 10 estimated, unconstrained
|
| for each row m:
|
v
_to_v15_constrained_nt(u) :: NamedTuple{(tau_F, B_inf, F_inf, lambda_A,
                                         mu_0, mu_B, mu_F,
                                         sigma_B, sigma_F, sigma_A), ...}
| exp(clamp(u, -20, 20))
v
fill_pinned_nt(estimated)            # add tau_B, eta, epsilon_A, mu_FF
| → NamedTuple of 14 v1.5 fields
v
params_v15_to_v1_nt(p)                # rotate B_inf → kappa_B, etc.
| → NamedTuple of 14 v1 fields
v
_pack_v1_row!(row, p_v1)              # write into params_per_chain[m, 1:14]
|
v
copyto!(target.params_per_chain[1:M, :], CPU rows)
|
v
copyto!(target.Phi_seq, target.obs_*) # stage obs onto GPU
|
v

```

```

fill!(bufs.particles_a, B_init/F_init/A_init); fill!(bufs.log_w, 0)
|
v
for seg in 1:n_segments:
|
|   propagate_segment_kernel!(..., Ntot=M*K, R=4, seg_step_offset)
|       per-thread: drift + diff + boundary + 3-channel Gaussian log-weight
|
|   (if not last) run_segmented_smc_step!(bufs, M, ...)    FRAMEWORK
|       per-chain resample + Liu-West + OT
|
|   stats kernel populates log_max, log_z, mu_per_chain
|
v
log_lik_acc[m] += log_max[m] + log_z[m] - log(K)    # CPU side
|
v
return log_lik_acc :: Vector{Float64}(M)            # PURE OUTPUT

```

### 5.3 Controller dataflow (one controller\_plan call)

```

posterior theta_dyn (filter posterior mean, v1.5 NamedTuple)
|
v
fill_pinned_nt(params_post_v15)                    # add tau_B, eta, epsilon_A, mu_FF
|
v
params_v15_to_v1_nt(p_v15)                        # to v1 form
|
v
FSAv1ControlGPUPTarget(; n_inner, M_max, n_steps=H_plan_bins,
                        n_anchors=8, dt, F_max=0.40, Phi_max=3.0,
                        Phi_default=1.0, lam_F=1.0, sigma_prior,
                        params=p_v1, init_state=plant_xhat,
                        noise_seed)                # constructor
|
v
make_log_density_fn(ctrl_target)                    # closure for the framework
|
v
run_tempered_smc_gpu(log_density_fn, M_max, n_smc, n_anchors,
                    0.0, sigma_prior, rng;
                    target_nats, ..., chees_L_candidates,
                    hmc_step_size, hmc_num_leapfrog,
                    num_mcmc_steps, max_temp_levels)    FRAMEWORK
|   returns U_ctrl (n_smc, n_anchors), n_temp, beta_max
v
theta_ctrl = mean(U_ctrl; dims=1)                  # posterior-mean RBF coefficients
|
v
RBF decode: for k in 1:T_total_bins:
    raw = c_Phi + Sum_a theta_a * exp(-0.5 * ((t_k - anchor_a)/sigma_rbf)^2)

```

```

        |
        |
        v
return (Phi_plan, n_temp_ctrl, theta) :: NamedTuple

```

## 6 CLI flags reference for bench\_smc\_full\_mpc\_fsa\_gpu.jl

The bench has 23 CLI flags total: 14 for the bench / filter, 9 for the controller (all newly CLI-exposed in this version of v1.5).

### 6.1 Bench / filter flags

| Flag              | Default | Type   | Meaning                                                                                                                                                                                                                         |
|-------------------|---------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| --T-days          | 14      | Int    | Total bench horizon in days. The plant rolls forward this many days under the controller's schedule.                                                                                                                            |
| --step-minutes    | 60      | Int    | Per-bin time step in minutes; sets <code>BINS_PER_DAY = 1440 / step-minutes</code> . Must be set BEFORE the model imports (the bench exports <code>ENV["FSA_STEP_MINUTES"]</code> ).                                            |
| --replan-K        | 2       | Int    | Replan every $K$ strides at end-of-stride. Ignored when <code>--open-loop true</code> .                                                                                                                                         |
| --N-smc           | 32      | Int    | Filter outer-SMC <sup>2</sup> particles (number of $\theta$ -cloud chains).                                                                                                                                                     |
| --K-per-chain     | 200     | Int    | Inner-PF state particles per filter chain. Total filter particles = <code>N-smc · K-per-chain</code> .                                                                                                                          |
| --num-mcmc        | 3       | Int    | HMC moves per filter tempering level.                                                                                                                                                                                           |
| --hmc-step-size   | 0.05    | Float  | Filter HMC leapfrog step size.                                                                                                                                                                                                  |
| --hmc-leapfrog    | 4       | Int    | Filter HMC leapfrog trajectory length (number of leapfrog steps).                                                                                                                                                               |
| --max-lambda-inc  | 0.20    | Float  | Max tempering increment $\delta\lambda$ per filter tempering level.                                                                                                                                                             |
| --target-ess-frac | 0.5     | Float  | Target effective-sample-size fraction for tempering bisection.                                                                                                                                                                  |
| --max-temp-levels | 30      | Int    | Hard cap on filter tempering levels per stride.                                                                                                                                                                                 |
| --seed            | 42      | Int    | Master RNG seed. All sub-keys are derived from this via <code>hash((seed, :purpose, idx))</code> .                                                                                                                              |
| --output-dir      | ""      | String | Output directory. Empty $\Rightarrow$ auto-named under <code>outputs/fsa_high_res/g4_runs/</code> .                                                                                                                             |
| --open-loop       | "false" | String | "true" $\Rightarrow$ one initial controller plan from <code>INIT.STATE+TRUTH.PARAMS</code> , applied for the whole bench; no replanning. "false" $\Rightarrow$ closed-loop with replanning every <code>replan-K</code> strides. |

### 6.2 Controller flags (CLI-exposed, formerly hardcoded)

| Flag           | Default | Type | Meaning                                                                              |
|----------------|---------|------|--------------------------------------------------------------------------------------|
| --ctrl-n-smc   | 256     | Int  | Controller outer-SMC <sup>2</sup> particles ( $\theta_{\text{ctrl}}$ prior samples). |
| --ctrl-n-inner | 64      | Int  | Common-random-numbers MC trials averaged per cost evaluation.                        |

|                                 |     |       |                                                                                                                         |
|---------------------------------|-----|-------|-------------------------------------------------------------------------------------------------------------------------|
| <code>--ctrl-num-mcmc</code>    | 8   | Int   | ChEES-HMC moves per controller tempering level.                                                                         |
| <code>--ctrl-hmc-step</code>    | 0.2 | Float | Base HMC leapfrog step size for the controller.                                                                         |
| <code>--ctrl-hmc-leap</code>    | 16  | Int   | Base leapfrog trajectory length for the controller.                                                                     |
| <code>--ctrl-chees-max</code>   | 256 | Int   | ChEES picker upper bound. The candidate list is built as $[16, 32, 64, \dots, \text{max}]$ (powers of 2 up to the cap). |
| <code>--ctrl-max-levels</code>  | 25  | Int   | Hard cap on controller tempering levels per replan.                                                                     |
| <code>--ctrl-target-nats</code> | 8.0 | Float | Target nat budget for controller $\beta_{\text{max}}$ auto-calibration.                                                 |
| <code>--ctrl-sigma-prior</code> | 1.5 | Float | Prior std on $\theta_{\text{ctrl}}$ RBF coefficients.                                                                   |

---

### 6.3 Example invocations

**Open-loop diagnostic (3–4 min wall, T=14d):**

```
julia --project=. tools/bench_smc_full_mpc_fsa_gpu.jl \
  --T-days 14 --step-minutes 60 --open-loop true \
  --N-smc 8 --K-per-chain 100 \
  --num-mcmc 1 --hmc-leapfrog 2 --max-lambda-inc 0.30
```

**Closed-loop strong-controller (10–15 min wall, T=28d):**

```
julia --project=. tools/bench_smc_full_mpc_fsa_gpu.jl \
  --T-days 28 --step-minutes 60 --replan-K 2 \
  --N-smc 8 --K-per-chain 100 \
  --num-mcmc 1 --hmc-leapfrog 2 --max-lambda-inc 0.30 \
  --ctrl-n-smc 256 --ctrl-n-inner 64 \
  --ctrl-num-mcmc 8 --ctrl-hmc-leap 16 \
  --ctrl-chees-max 256
```

## 7 Lean4-port mapping appendix

This appendix exists because v1.5’s use of `Match.jl` is motivated by a future port to Lean4 for formal verification. Lean4’s `match ... with` syntax is structurally near-identical to `Match.jl`’s `@match ... begin ... end`, so writing the Julia in pattern-matching idioms now makes the future port nearly mechanical at every pattern site.

### 7.1 Side-by-side mappings

```
params_v15_to_v1_nt — Dict — NamedTuple dispatch.
1 # Julia (Match.jl)
2 params_v15_to_v1_nt(p) = @match p begin
3   ::Dict      => (kappa_B = p[:B_inf] / p[:tau_B], ...)
4   ::NamedTuple => (kappa_B = p.B_inf / p.tau_B, ...)
5 end
6
7 -- Lean4 (planned; sketch)
8 inductive ParamsForm where
9   | dictForm : (key -> Float) -> ParamsForm
10  | nt       : V15Params -> ParamsForm
11
12 def params_v15_to_v1 (p : ParamsForm) : V1Params :=
13   match p with
14   | .dictForm d => { kappa_B := d[:B_inf] / d[:tau_B], ... }
15   | .nt nt      => { kappa_B := nt.B_inf / nt.tau_B, ... }
```

#### `_reflect_unit` — boundary reflection.

```
1 # Julia (Match.jl)
2 _reflect_unit(x) = @match x begin
3     x, if x < 0 end => -x
4     x, if x > 1 end => 2 - x
5     -                => x
6 end
7
8 -- Lean4 (planned)
9 def reflect_unit (x : Float) : Float :=
10     match x with
11     | x => if x < 0 then -x
12           else if x > 1 then 2 - x
13           else x
```

#### `to_v15_constrained_nt` — vector destructure.

```
1 # Julia (Match.jl)
2 _to_v15_constrained_nt(u) = @match u begin
3     [u1, u2, u3, u4, u5, u6, u7, u8, u9, u10] =>
4         (tau_F = exp(clamp(u1, -20, 20)),
5          B_inf = exp(clamp(u2, -20, 20)),
6          ..., sigma_A = exp(clamp(u10, -20, 20)))
7 end
8
9 -- Lean4 (planned)
10 def to_v15_constrained (u : Vector Float 10) : V15Params :=
11     match u with
12     | #[u1, u2, u3, u4, u5, u6, u7, u8, u9, u10] =>
13         { tau_F := Float.exp (Float.min 20 (Float.max (-20) u1)),
14           B_inf := ...
15           sigma_A := ... }
```

#### `gpu_log_density` — last-segment branch.

```
1 # Julia (Match.jl)
2 @match (seg, n_segments) begin
3     (s, n) where s == n => begin
4         # last segment: stats kernel only (no resample)
5         ...
6     end
7     _ => begin
8         run_segmented_smc_step!(bufs, M, ...)
9         ...
10    end
11 end
12
13 -- Lean4 (planned)
14 match (seg, n_segments) with
15 | (s, n) => if s == n then
16     -- last segment: stats only
17     ...
18 else
19     -- run segmented SMC step + accumulate
20     ...
```

#### `parallel_hmc_one_move` — accept/reject.

```
1 # Julia (Match.jl)
2 @inbounds for m in 1:M
3     outcome = log(rand(rng)) < log_alpha[m] ? :accept : :reject
4     @match outcome begin
5         :accept => begin U_out[m, :] = U_new[m, :]; n_acc += 1 end
```

```

6         :reject => U_out[m, :] = U[m, :]
7     end
8 end
9
10 -- Lean4 (planned)
11 inductive HMCOutcome where | accept | reject
12
13 def hmc_step (m : Nat) (log_alpha u : Float) :=
14     let outcome := if Float.log u < log_alpha
15                     then .accept else .reject
16     match outcome with
17     | .accept => ... -- write U_new[m] into U_out[m], increment counter
18     | .reject => ... -- write U[m] into U_out[m]

```

```

apply_prior — :LogNormal — :Normal dispatch.
1 # Julia (Match.jl)
2 function apply_prior(kind, mu, sigma, u)
3     @match kind begin
4         :LogNormal => exp(clamp(u, -20, 20))
5         :Normal    => mu + sigma * u
6     end
7 end
8
9 -- Lean4 (planned)
10 inductive PriorKind where | logNormal | normal
11
12 def apply_prior (kind : PriorKind) (mu sigma u : Float) : Float :=
13     match kind with
14     | .logNormal => Float.exp (Float.min 20 (Float.max (-20) u))
15     | .normal    => mu + sigma * u

```

## 7.2 Type-system gotchas for the future Lean4 port

- **Float precision.** Julia’s Float64 maps to Lean4’s Float. Lean4 has no Float32; the GPU kernel’s fp32 paths cannot be ported directly — they would need a separate verified-arithmetic library or kept as Julia.
- **Mutable arrays.** v1.5’s pure facade returns new Vectors; Lean4’s idiomatic equivalent is to return a fresh `Array Float n`. The functional-only design of v1.5 is what makes the port cleanly possible without re-architecting.
- **Records vs inductives.** Julia’s NamedTuple maps to Lean4’s structure. The two-method `params_v15_to_v1_nt` becomes a single function over an inductive sum type `ParamsForm`. The `@match` version of the Julia function makes this transition transparent.
- **Out of scope for this round.** The GPU kernels (`propagate_segment_kernel!`, `fsa_v1_cost_kernel!`) are NOT on the Lean4 port path. Formal verification of fp32 GPU kernels is a separate, much harder problem (would need a verified-fp32 axiomatisation, KernelAbstractions → Lean4 lowering, etc.).

## 7.3 Verification of the port (when it happens)

The port is mechanical at every `@match` site. Validation will run the same FIM gate, the same psim sanity test, and the same 6-test smoke suite (`tools/test_gpu_pf.jl`) but in the Lean4-compiled native code. The `Match.jl` idioms make this process a series of local rewrites rather than a global re-architecture.